

[Day 1] 종합 실습 — 실행결과 정리

작성자 : 신주용 (광주 3반) · 실습명 : 실무형 수집·검증·품질 파이프라인 · 작성일 : 2026-07-20

1. 실행결과 화면 캡처

```
main.py · pytest · ruff - 실행 결과

$ python main.py

1) 비동기 수집 완료 : weather / country / ip 3종
2) 날씨 검증 : 정상 72건 / 오류 0건
   국가 : Korea (Republic of) / 수도 Seoul / 인구 51,780,579
   IP 8.8.8.8 : United States/Ashburn (America/New_York)
3) 저장/성능 비교 (weather 데이터):
   csv      | 쓰기 0.178ms | 읽기 0.203ms | 크기 1849B
   parquet | 쓰기 0.265ms | 읽기 0.429ms | 크기 3398B
파이프라인 완료

$ pytest -q
..... [100%]
5 passed in 0.04s

$ ruff check .
All checks passed!
```

3개 공개 API(Open-Meteo · Countries.dev · ip-api) 비동기 동시 수집 → Pydantic v2 검증
→ CSV · Parquet 저장·성능 비교 결과.
pytest 5건 통과 · ruff 오류 0건 확인. (날씨 · IP 값은 실시간 API 라 실행 시점마다 변동.)

2. 코드 분석 결과에 대한 본인 의견

수집 · 검증 · 저장 3단계 구현·실행하며 확인한 점 정리.

1. 비동기 수집 효과

순차 호출 시 네트워크 대기시간 그대로 합산. asyncio.gather 동시 수집으로 전체 대기를 가장 느린 1건 수준까지 단축. 수집 대상 늘수록 이득 확대되는 구조.

2. 선언적 검증의 장점

Pydantic v2 Field 제약(ge/le·gt·min_length·Literal)으로 타입·범위를 선언적으로 검증. 수동 if 검증 코드 제거, 오류 메시지도 표준 형식으로 자동 생성 → 유지보수 용이.

3. CSV vs Parquet 해석

소량(72행)에서는 CSV 가 쓰기·읽기·용량 모두 우세. Parquet 는 컬럼 압축·스키마 보존이 강점이라 대용량·반복 조회에서 우위. 최적 형식은 데이터 규모에 좌우 → 소량은 CSV, 대용량은 Parquet.

4. 측정의 신뢰성

첫 호출은 엔진(pyarrow) 초기화 비용 혼입으로 수백 ms 발생. 워밍업 1회 제외 + best-of-5 최소값 측정 → 형식 간 비교 왜곡 방지.

5. 예외 처리 설계

수집 단계는 return_exceptions 로 일부 실패해도 나머지 확보 후 실패 API 재통보. 검증 단계는 ValidationError·KeyError 분리 처리 → 실패 원인 명확화.

3. 추가 의견 (개선사항 · 코드 품질 개선 측면)

현행으로도 요구사항 충족. 실무 수준 향상 시 아래 보완 가능.

1. 재시도 정책

현재 timeout 10초 실패 시 즉시 중단. 지수 백오프 재시도(tenacity 등) 도입 시 일시적 네트워크 오류 흡수 → 견고성 향상.

2. 응답 캐싱

국가 정보 등 저변동 응답은 TTL 로컬 캐시로 관리 → 불필요한 재호출 감소.

3. 설정 외부화

API URL·좌표·타임아웃을 코드 상수 대신 설정파일(.env/pyproject)로 분리 → 재사용성·테스트 용이성 향상.

4. 로깅 전환

print 대신 logging 모듈 레벨별(INFO/ERROR) 기록 → 운영 환경 관측성 확보.

5. 테스트 확장

현재 모델 단위 테스트 중심. httpx mock(respx)으로 collector 통합 테스트 추가 시 네트워크 없이 수집 로직까지 검증 가능.

6. 스키마 진화 대비

응답 필드 추가·변경 대비해 모델에 extra 정책(ignore/forbid) 명시 → API 변화에 안전.